

OPERATING SYSTEMS

Process Coordination & Deadlocks

S.Thenmozhi

Department of Computer Applications

- **Key idea:** Restrain the ways request for resources can be made
- Recall, all 4 necessary conditions must occur for a deadlock to happen
 1. Mutual Exclusion
 2. Hold and Wait
 3. No preemption
 4. Circular Wait
- If we ensure at least one condition is not met, we **prevent** a deadlock

- **Mutual Exclusion** – **not required for sharable resources**; must hold for non-sharable resources
- **Hold and Wait** – must guarantee that **whenever a process requests a resource, it does not hold any other resources**
- **No Preemption** –
 - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all **resources currently being held are released**
 - Process are restarted to gain the resources held by them
- **Circular Wait** – impose a total ordering of all resource types, **and processes requesting resources in an increasing order**

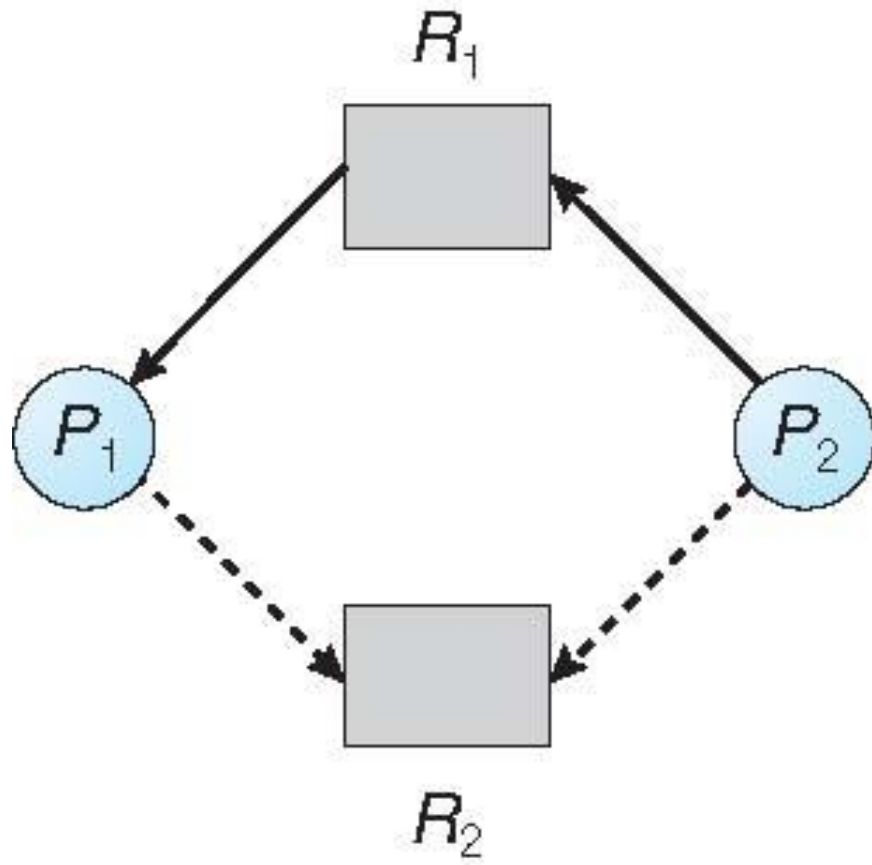
- **Idea: Allocate the resources only if the system in safe state**
- When you are allocating a resource to the process, check whether the system remains in safe state or unsafe state.
- System is in **safe state** if there exists a sequence $\langle P_1, P_2, \dots, P_n \rangle$ of ALL the processes in the systems such that for each P_i , the resources that P_i requests can be satisfied by currently available resources + resources held by all the other processes P_j
- Basic Facts
 - If a system is in safe state $\Rightarrow$ no deadlocks
 - If a system is in unsafe state $\Rightarrow$ possibility of deadlock
 - Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state

OPERATING SYSTEMS

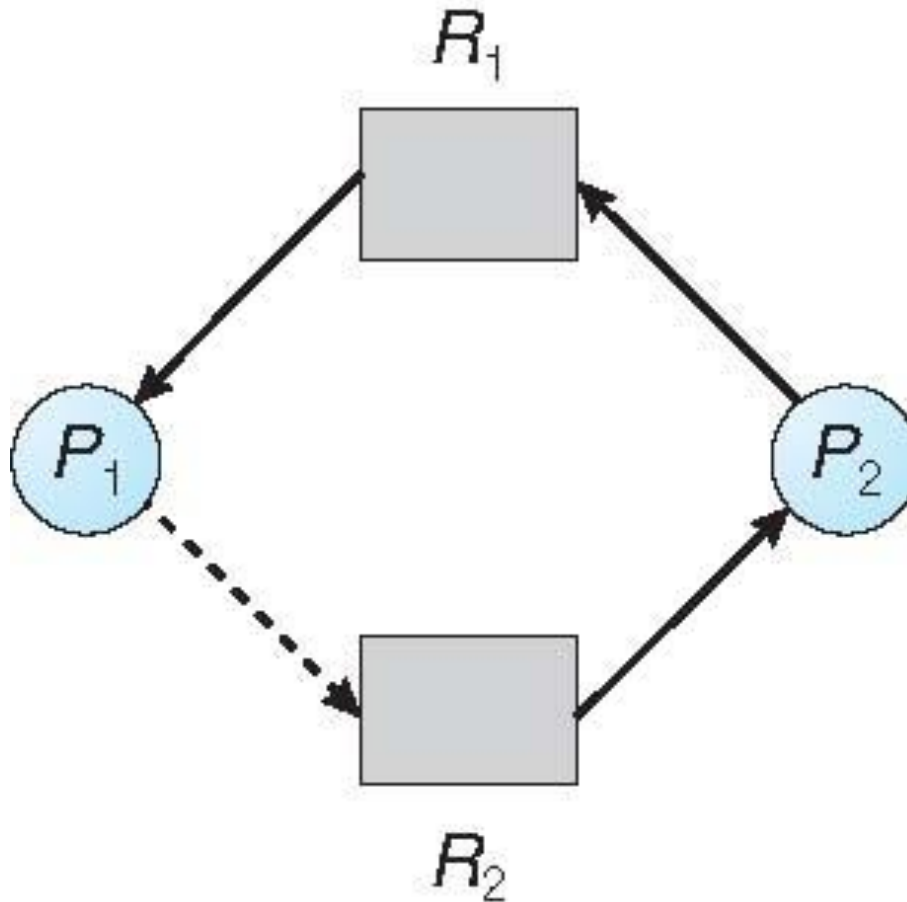
Deadlock Avoidance

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm

- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_i **may request** resource R_j ; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system



RAG - Unsafe State





THANK YOU

S. Thenmozhi

Department of Computer Applications

thenmozhis@pes.edu

+91 80 6666 3333 Extn 393